tmux - terminal multiplexer

Prefix = Ctrl+a (press, release, then the key)

Basics

Ctrl+a	prefix key
Ctrl+a a	send a literal Ctrl+a to the app
r	reload ~/.tmux.conf
?	list all key bindings
:	tmux command prompt
d	detach (everything keeps running)
I / U	plugins: install / update (TPM)
Ctrl+s / Ctrl+r	save / restore sessions (resurrect)

Windows (tabs)

c	new window in the current folder
1 .. 9	go to window number
n / p	next / previous window
Ctrl+h / Ctrl+l	previous / next (repeatable)
Tab	jump to the last window
,	rename window
&	close window
w	tree of all windows and sessions

Sessions

S	fuzzy session switcher (fzf popup)
s	session list (built-in)
$	rename session
(/)	previous / next session

Panes (splits)

|	split left | right
-	split top / bottom
h j k l	move left / down / up / right
H J K L	resize (hold to repeat)
z	zoom pane full-screen (toggle)
x	close pane
q	show pane numbers
Space	cycle layouts
{ / }	swap pane up / down
!	move pane into its own window

Without prefix

Ctrl+h j k l	move between tmux panes AND vim splits (vim-tmux-navigator)
Mouse click	select pane / window
Drag border	resize pane
Wheel	scroll history (enters copy mode)
Drag text	select; copied to system clipboard
Double-click	select word

Clear screen: prefix then Ctrl+l (Ctrl+l alone is used by the vim navigator). macOS: hold Option while dragging for iTerm2's own selection.

Command line

tmux new -s NAME	new named session
tmux ls	list sessions
tmux attach -t NAME	attach to a session
tmux kill-session -t NAME	kill a session
tmux kill-server	kill everything
NO_TMUX=1 zsh	skip tmux autostart once

Copy mode (vim keys)

[or Esc	enter copy mode (or scroll with the wheel)
h j k l w b	move like vim
gg / G	top / bottom of history
Ctrl+u / Ctrl+d	half page up / down
/ ?	search forward / backward
n / N	next / previous match
v	start selection
V	select whole line
Ctrl+v	rectangular selection
H / L	start / end of line
y	copy to clipboard and exit
q	leave copy mode
P	paste (prefix + P)

Popups and tools

g	lazygit in a popup (current folder)
b	btop in a popup
S	fzf session switcher
D	workmux dashboard (see page 2)

Aliases (zsh): wm = workmux, lg = lazygit; oh-my-zsh tmux plugin adds ta, tl, ts, tad.

Config files

~/.tmux.conf	main config
~/.tmux.agents.conf	workmux key bindings
~/.tmux/plugins/	TPM plugins

workmux + Pi - parallel AI agents

Run inside tmux, inside a git repo

workmux: create a task (worktree + tmux window + Pi)

`workmux add NAME`	new branch, worktree, window; Pi starts
`-p "prompt"`	start Pi with this prompt
`-P file.md`	prompt from a file
`-A -e`	write prompt in $EDITOR; branch name auto-made (0 tokens)
`-b`	create in background (don't switch)
`-l solo | dev`	layout: agent only / agent+shell+lazygit
`-w`	move uncommitted changes into the new task
`--base BRANCH`	branch from BRANCH (default: main)
`--pr 123`	check out a pull request
`-a pi -n 3`	3 parallel copies of the same task
`-o`	open it if it already exists

workmux: watch

`workmux list (ls)`	all worktrees, agent status, unmerged
`workmux status`	agent status for scripts
`workmux dashboard`	live TUI of all agents
`workmux sidebar`	toggle status sidebar
`workmux capture NAME`	print an agent's terminal output
`workmux send NAME ".."`	send a message to a running agent
`workmux path NAME`	folder of a worktree

Everyday flow

1. Open the terminal: tmux starts by itself. cd into your repo.
2. prefix N (type a name) or prefix O (write a prompt), or workmux add NAME -p "...".
3. Start more tasks. Each gets its own branch and folder: no clashes.
4. prefix D to watch all; prefix u to jump to the agent that finished.
5. Review in the task window (prefix g = lazygit), then: workmux merge NAME.

workmux: finish and manage

`workmux merge NAME`	rebase, merge, then remove worktree, window, branch
`--squash`	squash into one commit
`--keep (-k)`	merge but keep worktree and window
`-n`	skip pre-merge checks
`workmux rm NAME`	remove without merging (-f = force)
`workmux rm --gone`	remove tasks whose remote branch is gone
`workmux rm --all -f`	remove all tasks
`workmux close NAME`	close window, keep worktree
`workmux open NAME`	reopen window for a worktree
`workmux rename / rebase`	rename task / rebase onto base
`workmux resurrect`	restore windows after a crash

workmux: tmux keys (prefix first)

`D`	dashboard popup (d diff, i input, Enter jump)
`Ctrl+w`	dashboard, worktrees tab
`T`	toggle sidebar
`u`	jump to agent that finished / waits
`A`	toggle between last two agents
`N`	new task: type a branch name
`O`	new task: write prompt in $EDITOR

Window names show status: working / done (Pi has no 'waiting' state).

workmux: setup and files

`workmux setup`	install Pi status hooks (run once in tmux)
`workmux init`	create .workmux.yaml in a repo
`workmux config edit`	edit the global config
`workmux config reference`	every option, documented
`workmux docs`	full documentation
`~/.config/workmux/`	config.yaml, slug-name.pl
`<repo>/.workmux.yaml`	per-project overrides
`<repo>__worktrees/NAME`	where a task's files live

Pi: keys

`Enter`	send; while working = steer the agent now
`Alt+Enter`	queue a follow-up for when it finishes
`Alt+Up`	pull queued message back to editor
`Esc`	abort / cancel
`Esc Esc`	open /tree (session history)
`Ctrl+C`	clear editor; twice = quit
`Ctrl+D`	quit when editor is empty
`Ctrl+L`	model selector
`Ctrl+P / Shift+Ctrl+P`	cycle scoped models fwd / back
`Shift+Tab`	cycle thinking level
`Ctrl+O / Ctrl+T`	fold tool output / thinking
`Ctrl+G`	write the prompt in $EDITOR
`Ctrl+V`	paste image or text
`!cmd / !!cmd`	run shell: send output / don't send

Pi: slash commands

`/login /logout`	Claude, Copilot, Mistral, ...
`/model`	switch model (Ctrl+S saves as default)
`/scoped-models`	models used by Ctrl+P
`/settings`	thinking, theme, delivery
`/resume /new`	old session / fresh session
`/tree /fork /clone`	branch the conversation
`/compact [hint]`	summarize old context
`/session /name X`	tokens and cost / rename
`/reload /hotkeys`	reload config / all keys
`/quit`	exit

Pi: files and tips

`~/.pi/agent/`	settings, models, auth, keybindings (.json)
`pi -p "text"`	one-shot, non-interactive
`Ollama`	OLLAMA_CONTEXT_LENGTH=32768 ollama serve

iTerm2: Left Option Key = Esc+ so Alt+Enter works. Never paste keys into prompts; auth.json should be chmod 600.